

English → Bhojpuri Low-Resource Neural Machine Translation

10-Hour Hackathon Execution Plan • 2 Members • PyTorch + HuggingFace + Gradio

Goal: Build a simple, reproducible English-to-Bhojpuri neural machine translation prototype using a limited parallel dataset, subword/BPE-style tokenization, a pretrained Seq2Seq Transformer, BLEU evaluation, and a Gradio demo.

1. Problem Statement in Simple Words

The project should translate English sentences into Bhojpuri even though Bhojpuri has fewer digital translation resources than major languages. Instead of training a large Transformer from scratch, the team will adapt a pretrained multilingual Seq2Seq model using a Bhojpuri parallel dataset. The system will be evaluated with BLEU and exposed through a lightweight Gradio interface.

2. Final Architecture

Dataset → cleaning/splitting → subword tokenization → pretrained multilingual Seq2Seq Transformer → fine-tuning → test translation → BLEU → Gradio → GitHub

Recommended practical approach: use a pretrained HuggingFace multilingual model rather than training a Transformer from zero. Keep the first successful run small; increase data/model settings only if time remains.

3. Suggested Stack

Component	Use
Language	Python
Deep learning	PyTorch
Model ecosystem	HuggingFace Transformers
Dataset handling	HuggingFace Datasets / pandas
Tokenization	Model tokenizer using SentencePiece/subword tokenization; document BPE/subword behavior
Evaluation	SacreBLEU or HuggingFace Evaluate
Demo	Gradio
Version control	Git + GitHub
Environment	Google Colab or local GPU

4. Team Split — Work Separately

Member 1 — ML / NLP	Member 2 — Evaluation / App / Docs
Dataset download + cleaning	Gradio interface
Train/validation/test split	Evaluation display
Tokenizer/model setup	Example test sentences
Fine-tuning	Charts / results table
Inference script	README + screenshots

BLEU calculation support	GitHub integration
Save model/checkpoint	Final integration

5. Member 1 — ML/NLP Workflow

Step 1 — Dataset

Use an English–Bhojpuri parallel dataset. Start with a manageable subset if compute/time is limited. Inspect columns and remove empty/duplicate pairs.

Step 2 — Clean

Normalize whitespace, remove unusable rows, verify that English and Bhojpuri sentences remain aligned, and save a cleaned CSV/JSONL.

Step 3 — Split

Use a reproducible split such as 80% train, 10% validation, 10% test. Set a random seed.

Step 4 — Tokenizer

Load the tokenizer belonging to the chosen pretrained multilingual Seq2Seq model. If the model uses SentencePiece/subword tokenization, explain this as the practical subword mechanism used for rare words.

Step 5 — Baseline

Run a very small baseline training/inference test first. Confirm that tokenization, labels, loss, and decoding work before launching a longer run.

Step 6 — Fine-tune

Fine-tune the pretrained model on English→Bhojpuri. Keep batch size and sequence length modest. Save the best checkpoint.

Step 7 — Inference

Write one clean function: `translate(text) → Bhojpuri text`. Test 10–20 unseen English sentences.

Step 8 — BLEU

Run the final model on the held-out test set and calculate corpus BLEU. Save the numeric result and the test configuration.

6. Member 1 — Minimum Files to Deliver

```
data/cleaned.csv • notebooks/01_data_check.ipynb • notebooks/02_train.ipynb • src/train.py • src/translate.py  
• src/evaluate.py • models/README.md • requirements.txt
```

7. Member 2 — Gradio + Evaluation Workflow

Step 1 — UI skeleton

Create a Gradio page with English input, Translate button, Bhojpuri output, model information, and BLEU result.

Step 2 — Translation hook

Initially connect a dummy function so the UI works. Later replace it with Member 1's `translate()` function.

Step 3 — Evaluation panel

Display the final corpus BLEU score and a small table of sample English/reference/predicted translations.

Step 4 — Error examples

Collect 5–10 examples where the output is strong and 3–5 examples where it differs from the reference. Do not hide failures.

Step 5 — Documentation

Write the README: problem, architecture, dataset, setup, training, evaluation, demo, limitations, and team contributions.

Step 6 — Final integration

Copy only the required model artifacts/configuration or use a saved checkpoint path. Test the complete app from a fresh environment.

8. Gradio UI — Keep It Simple

Title: Bhojpuri AI Translator

Input: English sentence

Button: Translate

Output: Bhojpuri translation

Info: Model name, training pairs used, BLEU score

Optional: Example buttons and translation history

9. 10-Hour Timeline

Hour	Member 1	Member 2	Checkpoint
0–1	Dataset + environment	Repo + UI skeleton	Both can run locally/Colab
1–2	Clean + split data	UI fields + dummy translation	Clean dataset ready
2–3	Tokenizer + model test	Evaluation table design	One sample inference works
3–5	Fine-tune first run	README + sample data	First checkpoint
5–6	Inference + BLEU	Gradio integration	Model works in UI
6–7	Improve training if needed	UI polish + results	Stable demo
7–8	Final test + save artifacts	Final docs + screenshots	Integration complete
8–9	Help with presentation	GitHub cleanup + PPT	Release candidate
9–10	Final validation	Final validation	GitHub + demo ready

10. GitHub Repository Structure

```
bhojpuri-nmt/ ■■■ data/ ■■■ README.md ■■■ notebooks/ ■■■ 01_data_check.ipynb ■■■ 02_train.ipynb ■■■  
src/ ■■■ train.py ■■■ translate.py ■■■ evaluate.py ■■■ app.py ■■■ models/ ■■■ README.md ■■■  
results/ ■■■ bleu.txt ■■■ sample_translations.csv ■■■ results.png ■■■ screenshots/ ■■■  
requirements.txt ■■■ README.md ■■■ .gitignore
```

11. Git Workflow — Prevent Conflicts

Start: Create one GitHub repository and agree on the folder structure before coding. Each member works mainly on separate files/folders. Member 1 handles src/train.py, translate.py and data/notebooks; Member 2 handles app.py, results, screenshots and README.

```
git clone cd bhojpuri-nmt git checkout -b member1-ml # or git checkout -b member2-ui git add . git commit -m  
"add training pipeline" git push -u origin member1-ml
```

Final integration: merge the two branches only after both members commit. Resolve conflicts together. Run the complete application after merging.

12. What Must Be in the Final Repository

- Working training/fine-tuning code.
- Clear dataset source and preprocessing description. Do not commit a large dataset if its license/distribution terms do not allow it.
- Tokenizer/model configuration.
- Held-out BLEU result with dataset split and test-set details.
- At least 10 sample translations.
- Gradio app that actually loads the trained model/checkpoint.
- requirements.txt and setup instructions.
- README with architecture diagram and team contributions.
- Limitations and future work.

13. BLEU Evaluation Protocol

Use a fixed held-out test set that was never used for training. Report the number of test sentences, preprocessing/tokenization choices, BLEU implementation, and the final score. Do not compare your BLEU score to another model unless the datasets, preprocessing, language direction, and evaluation procedure are comparable.

14. Low-Resource Experiment — Strong Optional Feature

If training time permits, train on progressively larger subsets such as 2K, 5K, 10K, and 20K sentence pairs while keeping the validation/test sets fixed. Plot training-data size against BLEU. This directly demonstrates the project's low-resource focus. If time is short, run only 2K vs 10K.

15. What to Explain to Judges

Question	Short answer
Why Bhojpuri?	It is a regional language with fewer MT resources than major languages, making it appropriate for a low-resource
Why a pretrained model?	It transfers multilingual knowledge and reduces the data/compute required compared with training from scratch.
Why BPE/subwords?	Subword units reduce the impact of rare and unseen words and keep the vocabulary manageable.
Why attention/Transformer?	Attention lets the model use relevant context while encoding and generating sequences; Transformers are effective
Why BLEU?	It provides a standard automatic corpus-level comparison against reference translations, while not being a complete
What is the limitation?	Small/biased training data, spelling variation, domain mismatch, and BLEU's limitations can affect results.

16. Final 30-Minute Checklist

- Model loads without errors.
- Translation works for at least 10 unseen inputs.
- BLEU score is calculated from a held-out test set.
- Gradio app works from the same environment described in README.
- requirements.txt is complete.
- Dataset source/license is documented.
- No secrets/API keys are committed.
- Large model/data files are handled appropriately (Git LFS or documented download step if needed).
- README contains setup + run commands.
- GitHub repository has both members' contributions.
- Final demo and presentation use the same model/checkpoint as the submitted code.

17. Recommended Scope if You Fall Behind

Cut in this order: multiple training-size experiments → fancy UI → charts → extra model comparisons. **Never cut:** working translation, reproducible training/inference code, held-out BLEU evaluation, README, and a working Gradio demo.

18. Suggested Final Project Statement

“We developed a low-resource English-to-Bhojpuri neural machine translation system by adapting a pretrained multilingual Seq2Seq Transformer to a limited parallel corpus. The system uses subword tokenization, attention-based Transformer encoding/decoding, fine-tuning, held-out BLEU evaluation, and a Gradio interface for interactive translation.”

Keep the implementation small, reproducible, and demo-ready. The strongest use of the 10 hours is a complete end-to-end pipeline that actually runs.